

Diseño de la arquitectura de una red social

**Trabajo Arquitecturas Software
Ingeniería de Software I. Grupo 15.
Curso 2025-2026**

Haodi Lin Sun

Hecho con compañeros

Madrid, 10 de abril de 2026

Versión 202604061052

1. INTRODUCCIÓN

1.1. ÍNDICE

1. INTRODUCCIÓN
 - 1.1. Índice
 - 1.2. Objetivo del trabajo, enfoque y metodología
2. DRIVERS ARQUITECTÓNICOS
 - 2.1. Drivers funcionales
 - 2.2. Atributos de calidad
 - 2.3. Restricciones
3. INTERFACES DEL SISTEMA (APIs)
 - 3.1. Diagrama de secuencia
 - 3.2. Recursos y métodos
 - 3.3. Ejemplo de Información de recursos (formato JSON)
4. VISTA GENERAL DE LA ARQUITECTURA (MODELO ARQUITECTÓNICO)
5. COMPONENTES DE LA ARQUITECTURA Y DECISIONES DE DISEÑO
6. CONCLUSIONES

1.2. OBJETIVO DEL TRABAJO, ENFOQUE Y METODOLOGÍA

El presente trabajo consiste en el diseño de la arquitectura de un sistema software aplicando el proceso para el desarrollo de arquitecturas software seguido en clase y utilizando un estilo arquitectónico basado en microservicios. También, se deberán utilizar los componentes y patrones arquitectónicos vistos en clase de la asignatura.

El objetivo final es el diseño de la arquitectura software de una red social (tipo Facebook) donde el problema que el cliente quiere resolver puede definirse como “Un sistema que permita a diferentes usuarios publicar contenido multimedia (texto, imágenes, videos) en su muro. Sus amigos pueden dar "me gusta" o responder a esas publicaciones, según las vean”.

Para llevar a cabo el trabajo de diseño de la arquitectura software, se contemplarán las siguientes pautas:

- Se deberá diseñar la arquitectura del sistema asignado identificando al menos cinco drivers funcionales, tres drivers atributo de calidad y una restricción (pueden identificarse más si se considera necesario). Se deberán elegir los drivers arquitectónicos que se consideren más relevantes para el sistema a diseñar.
- Durante el diseño de la arquitectura se deberá seguir el proceso de diseño de una arquitectura software visto en clase, compuesto por tres pasos bien marcados: identificar los drivers de la arquitectura, definir las interfaces del sistema y crear la estructura de la arquitectura.
- La arquitectura propuesta deberá estar basada en microservicios. Para el diseño de la arquitectura se deberán aplicar los conceptos de arquitecturas vistos en clase: componentes y patrones arquitectónicos.
- La arquitectura deberá documentarse siguiendo la plantilla asociada a este documento. La memoria de la práctica tiene un límite de 10 páginas.
- Es importante describir adecuadamente las decisiones de diseño tomadas a lo largo del diseño de la arquitectura, así como los beneficios y posibles inconvenientes de cada decisión (trade-offs).

2. DRIVERS ARQUITECTÓNICOS

Asumimos que en este proyecto se ha llevado previamente a cabo una fase de Ingeniería de Requisitos de la que se ha obtenido un documento de Especificaciones de Requisitos de Software (SRS). Como extracto de ese documento se muestra aquí el listado de requerimientos funcionales:

ID	REQUISITO FUNCIONAL	DESCRIPCIÓN DEL REQUISITO FUNCIONAL
RFU.1	Registro de usuario	El sistema deberá permitir a un usuario registrarse en la plataforma proporcionando datos básicos como nombre, correo electrónico y contraseña.
RFU.2	Autenticación	El sistema deberá permitir a los usuarios iniciar sesión mediante sus credenciales para acceder a sus funcionalidades personales.
RFU.3	Gestión de perfil	El sistema deberá permitir a los usuarios crear y modificar su perfil, incluyendo información personal, foto y biografía.
RFU.4	Gestión de relaciones	El sistema deberá permitir a los usuarios seguir o agregar a otros usuarios como amigos, así como aceptar o rechazar solicitudes de amistad.
RFU.5	Publicación de contenido	El sistema deberá permitir a los usuarios crear, editar y eliminar publicaciones en formato multimedia, que podrán ser vistas por otros usuarios según su configuración de privacidad.
RFU.6	Interacción con publicaciones	El sistema deberá permitir a los usuarios comentar, reaccionar o compartir publicaciones realizadas por otros usuarios.
RFU.7	Mensajería privada	El sistema deberá permitir a los usuarios enviar y recibir mensajes privados entre ellos.
RFU.8	Configuración de privacidad	El sistema deberá permitir a los usuarios configurar quién puede ver su perfil y sus publicaciones
RFU.9	Alarmas de interés	El sistema deberá avisar a un usuario cuando en el muro se publique algo de potencial interés para él: reacción de otro usuario a un post suyo, mención, publicación sobre una materia de su interés...
RFA.1	Gestión de usuarios	El sistema deberá permitir al administrador consultar, modificar, suspender o eliminar cuentas de usuario.
RFA.2	Moderación de contenidos	El sistema deberá permitir al administrador revisar, ocultar o eliminar publicaciones que incumplan las normas de la plataforma.
RFA.3	Gestión de denuncias	El sistema deberá permitir al administrador consultar y gestionar las denuncias realizadas por los usuarios sobre contenidos o comportamientos inapropiados.
RFA.4	Gestión roles y permisos	El sistema deberá permitir al administrador asignar diferentes roles (administrador, moderador, usuario) y configurar sus permisos.
RFA.5	Gestión normas de la comunidad	El sistema deberá permitir al administrador definir y actualizar las políticas de uso y normas de la plataforma.
RFA.6	Estadísticas de uso	El sistema deberá permitir al administrador consultar estadísticas sobre número de usuarios, actividad en la red y publicaciones.
RFA.7	Gestión contenidos destacados	El sistema deberá permitir al administrador destacar o promocionar determinadas publicaciones o perfiles.

2.1. DRIVERS FUNCIONALES

Del conjunto de requisitos listados en el apartado anterior han sido seleccionados aquellos con una especial relevancia en el cumplimiento de los objetivos del negocio, cuya implementación pueda suponer una especial complejidad o aquellos que puedan exigir un escenario relevante para la arquitectura. Estos especiales requisitos funcionales seleccionamos que aquí denominamos drivers funcionales, junto a la justificación para ser considerados como tales, se detallan a continuación:

ID	REQUISITO DRIVER FUNCIONAL	DESCRIPCIÓN DEL DRIVER FUNCIONAL Y JUSTIFICACIÓN PARA DAR A ESTE REQUISITO RANGO DE DRIVER ARQUITECTÓNICO
RFU.2	Autenticación	El sistema deberá permitir a los usuarios iniciar sesión mediante sus credenciales para acceder a sus funcionalidades personales. <i>Justificación para dar a este requisito rango de driver arquitectónico: la autenticación es un requisito fundamental que condiciona la arquitectura del sistema porque determina cómo se gestionará la identidad de los usuarios y el acceso seguro a los servicios de la plataforma. Este requisito afecta directamente a decisiones arquitectónicas como la implementación de mecanismos de autenticación seguros, la gestión de sesiones, etc. Además, la autenticación puede impactar en otros atributos de calidad como seguridad, privacidad y control de acceso, que deben tenerse en cuenta en toda la arquitectura</i>
RFU.5	Publicación de contenido	El sistema deberá permitir a los usuarios crear, editar y eliminar publicaciones en formato multimedia, que podrán ser vistas por otros usuarios según su configuración de privacidad. <i>Justificación para dar a este requisito rango de driver arquitectónico: la publicación de contenido es una funcionalidad central de una red social y tiene un impacto directo en la arquitectura debido al alto volumen de datos y tráfico generado por los usuarios. Este requisito condiciona decisiones como el diseño de sistemas de almacenamiento escalables para contenido multimedia, el uso de servicios de almacenamiento distribuido o la arquitectura capaz de soportar altas tasas de escritura y lectura. Además, este requisito afecta a atributos de calidad como escalabilidad, rendimiento y disponibilidad, críticos en plataformas sociales</i>
RFU.6	Interacción con publicaciones	El sistema deberá permitir a los usuarios comentar, reaccionar o compartir publicaciones realizadas por otros usuarios. <i>Justificación como driver arquitectónico: las interacciones con publicaciones (comentarios, reacciones y comparticiones) generan una gran cantidad de eventos dentro del sistema y requieren una arquitectura capaz de procesar y propagar estas acciones casi en tiempo real. Este requisito también influye en otros drivers arquitectónicos, por ejemplo, atributos de calidad como rendimiento, escalabilidad y consistencia de datos.</i>
RFU.9	Alarmas de interés o notificaciones	El sistema deberá avisar a un usuario cuando en el muro se publique algo de potencial interés para él: reacción de otro usuario a un post suyo, mención, publicación sobre una materia de su interés, etc. <i>Justificación para dar a este requisito rango de driver arquitectónico: la existencia de un sistema de notificaciones requiere detectar eventos relevantes (comentarios, menciones, reacciones, etc.) y comunicar esa información a los usuarios de forma eficiente. Esto puede llevar a la necesidad de componentes arquitectónicos específicos como sistemas de notificaciones en tiempo real o servicios de procesamiento de eventos. Por otro lado, la funcionalidad de que un cliente sea notificado proactivamente cuando en el muro se ha producido algún cambio que le sea potencialmente interesante es un requerimiento muy ponderado en los análisis de mercado y la compañía lo considera una funcionalidad estratégica.</i>
RFA.1	Gestión de usuarios	El sistema deberá permitir al administrador consultar, modificar, suspender o eliminar cuentas de usuario. <i>Justificación para dar a este requisito rango de driver arquitectónico: La gestión de usuarios por parte del administrador es parte central del sistema y afecta a la arquitectura porque requiere mecanismos para administrar el ciclo de vida de las cuentas de usuario y poder aplicar políticas de control sobre ellas: gestión de roles y permisos, auditoría del sistema. mecanismos para bloqueo, suspensión o eliminación de cuentas, etc</i>
RFA.2	Moderación de contenidos	El sistema deberá permitir al administrador revisar, ocultar o eliminar publicaciones que incumplan las normas de la plataforma. <i>Justificación para dar a este requisito rango de driver arquitectónico: la moderación de contenidos requiere mecanismos que permitan revisar y gestionar publicaciones que incumplan las normas de la plataforma. Este requisito condiciona la arquitectura porque puede implicar: sistemas de revisión manual o automática de contenidos o integración con servicios de detección automática de contenido inapropiado.</i>
RFA.6	Estadísticas de uso	El sistema deberá permitir al administrador consultar estadísticas sobre número de usuarios, actividad en la red y publicaciones. <i>Justificación para dar a este requisito rango de driver arquitectónico: el análisis de estadísticas de uso requiere recopilar, almacenar y procesar grandes volúmenes de datos generados por la actividad de los usuarios. Este requisito obliga a decisiones arquitectónicas como implementación de sistemas de recopilación de eventos y métricas, uso de bases de datos analíticas o arquitecturas separadas para procesamiento analítico (analytics), así como generación de informes y dashboards para administración.</i>

2.2. DRIVERS ATRIBUTOS DE CALIDAD

2.2.1. SELECCIÓN Y JUSTIFICACIÓN DE LOS ATRIBUTOS DE CALIDAD

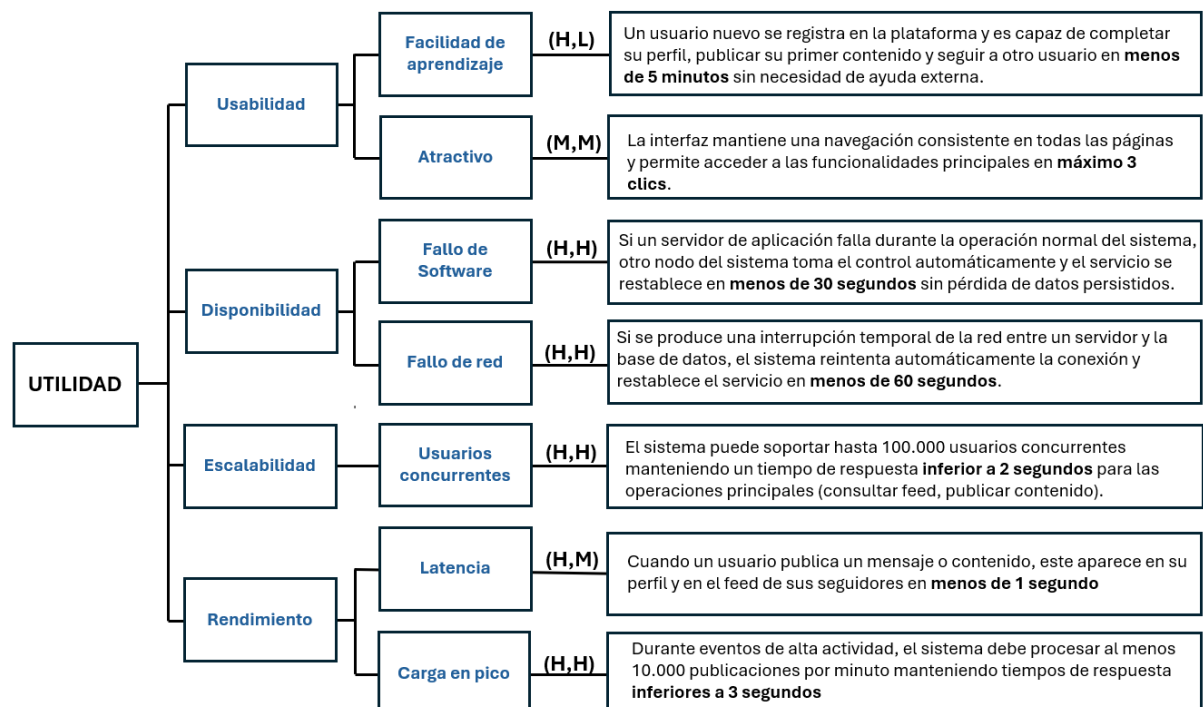
También del estudio del documento de Especificaciones de Requisitos de Software (SRS), emanan los requisitos funcionales no funcionales, de carácter cualitativo, con gran impacto estructural. En el caso de la red social que nos ocupa, asumimos como tales:

ID	ATRIBUTO DE CALIDAD	DESCRIPCIÓN DEL ATRIBUTO DE CALIDAD Y JUSTIFICACIÓN PARA DAR A ESTE ATRIBUTO RANGO DE DRIVER ARQUITECTÓNICO
ATC.1	Usabilidad	Se quiere una red social diferencialmente atractiva y fácil de participar en ella. Que pueda publicar contenidos de diversos formatos multimedia, fácil de aprender a usar y sencilla de operar. <i>Justificación para dar a este atributo de calidad el rango de driver arquitectónico: por razones de estrategia de marketing y competitividad, la red social proyectada se quiere que sea diferencial en esta funcionalidad respecto a otras redes sociales del mercado.</i>
ATC.2	Disponibilidad	Se quiere una red social, capaz de funcionar de forma continua 24x7 <i>Justificación para dar a este atributo de calidad el rango de driver arquitectónico: se considera que por las características del tipo de aplicación, de su uso y del perfil de los usuarios, la máxima disponibilidad debe ser considerado un “must” de diseño.</i>
ATC.3	Escalabilidad	Se quiere una red social capaz de soportar un importante crecimiento en el número de usuarios con una filosofía lineal respecto a la repercusión en el coste del crecimiento. <i>Justificación para dar a este atributo de calidad el rango de driver arquitectónico: es un mandato estratégico de la compañía la eficiencia y protección de la inversión tecnológica en el medio y largo plazo.</i>
ATC.4	Rendimiento	Se quiere una red social donde los usuarios tengan una percepción de inmediatez (de tiempo real) respecto a sus publicaciones o interacciones. <i>Justificación para dar a este atributo de calidad el rango de driver arquitectónico: la sensación de inmediatez ha sido un factor tan destacado en el estudio de mercado que se ha considerado como requerimiento de línea base (condición “sine qua non”).</i>

2.2.2. CUALIFICACIÓN DE LOS ATRIBUTOS DE CALIDAD. UTILITY MAP.

ÁRBOL DE UTILIDAD DEL SISTEMA

Identificación de los atributos de calidad prioritarios expresados en escenarios medibles y testeables



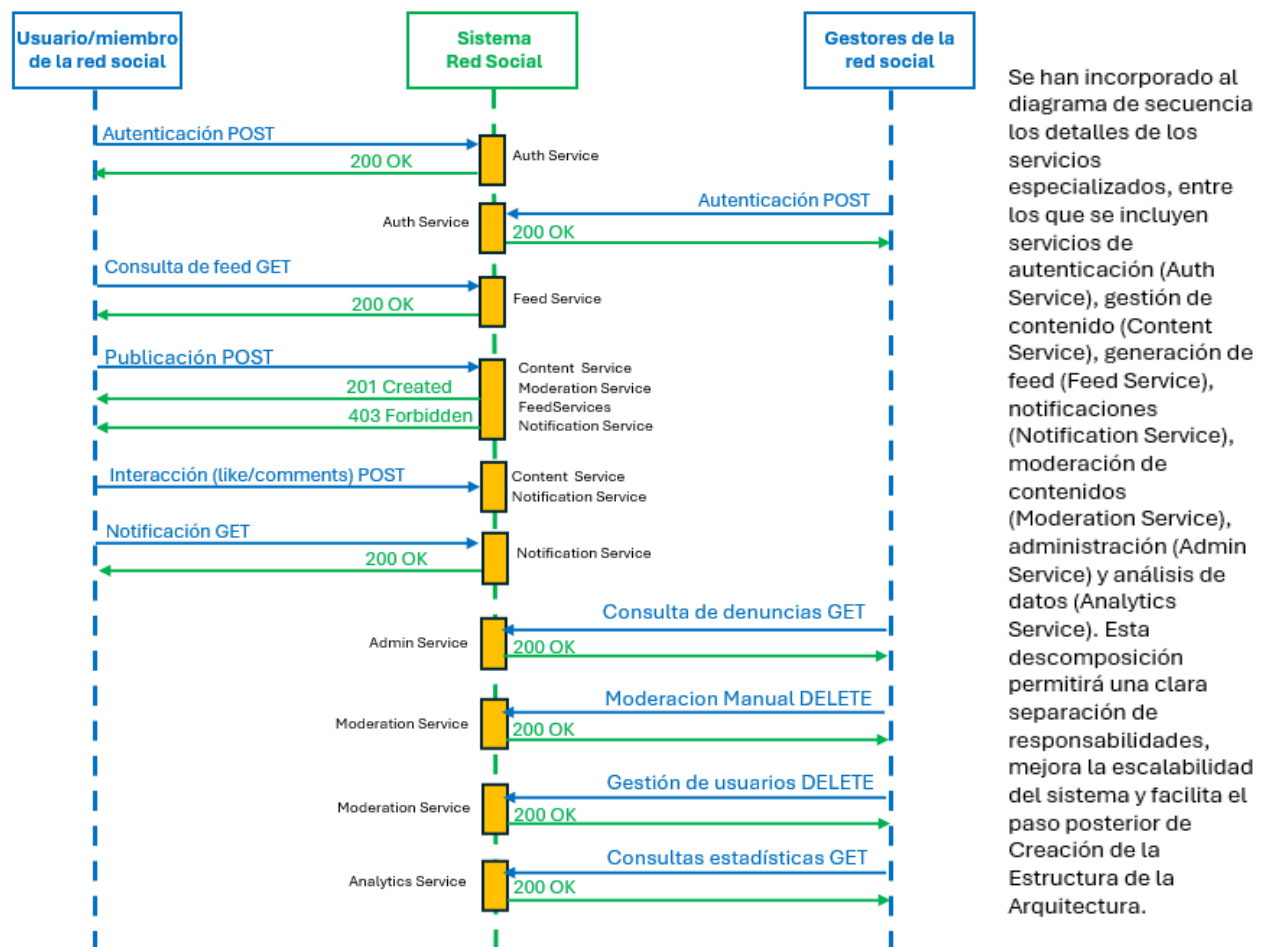
2.3. RESTRICCIONES

Asumimos en este proyecto como drivers arquitectónicos de restricción aquellas limitaciones que podrían condicionar la arquitectura del sistema. En el caso de la red social que nos ocupa, asumimos como restricciones:

ID	RESTRICCIÓN	DESCRIPCIÓN DE LA RESTRICCIÓN Y JUSTIFICACIÓN PARA DAR A ESTA RESTRICCIÓN EL RANGO DE DRIVER ARQUITECTÓNICO
RES.1	Restricción tecnológica. Arquitectura microservicios	El sistema deberá implementarse siguiendo una arquitectura basada en microservicios desplegada en la nube. <i>Justificación para dar a esta restricción el rango de driver arquitectónico: normativa tecnológica interna de la organización basada en experiencia y las buenas prácticas actuales en el dominio del comercio electrónico y plataformas digitales.</i>
RES.2	Restricción legal. Normativa europea.	El sistema deberá incorporar los mecanismos tecnológicos necesarios que permitan la detección, moderación y gestión de contenidos ilegales o dañinos, incluyendo discursos de odio, terrorismo o material inapropiado, así como mecanismos específicos de protección de menores <i>Justificación para dar a esta restricción el rango de driver arquitectónico: es un requerimiento legal del Reglamento Europeo de Servicios Digitales (Digital Services Act, DSA)</i>

3. INTERFACES DEL SISTEMA (APIs)

3.1. DIAGRAMA DE SECUENCIA



3.2. RECURSOS Y MÉTODOS

Se listan a continuación las Interfaces del sistema y sus características

Tabla de API's del Sistema			
Recursos	URIS	Métodos	Comentarios
1. Autenticación	/auth/register	POST	Registro de nuevos usuarios en la plataforma
2. Autenticación	/auth/login	POST	Autenticación de usuario y obtención de token JWT
3. Usuario	/users/{id}	GET	Obtener información del perfil de un usuario
4. Usuario	/users/{id}	PUT	Actualizar datos del perfil del usuario
5. Usuario	/users/{id}	DELETE	Eliminar cuenta de usuario
6. Relaciones	/users/{id}/follow	POST	Seguir a otro usuario
7. Relaciones	/users/{id}/follow	DELETE	Dejar de seguir a un usuario
8. Publicaciones	/posts	POST	Crear una nueva publicación
9. Publicaciones	/posts/{id}	GET	Obtener una publicación concreta
10. Publicaciones	/posts/{id}	PUT	Editar una publicación existente
11. Publicaciones	/posts/{id}	DELETE	Eliminar una publicación
12. Feed	/feed	GET	Obtener el feed personalizado del usuario. PAGINACIÓN
13. Likes	/posts/{id}/like	POST	Registrar un "me gusta" en una publicación
14. Likes	/posts/{id}/like	DELETE	Eliminar un "me gusta" de una publicación
15. Comentarios	/posts/{id}/comments	POST	Añadir un comentario a una publicación
16. Comentarios	/posts/{id}/comments	GET	Obtener los comentarios de una publicación
17. Notificaciones	/notifications	GET	Obtener las notificaciones del usuario. PAGINACIÓN
18. Notificaciones	/notifications/{id}/read	PUT	Marcar una notificación como leída
19. Administración	/admin/users	GET	Obtener listado de usuarios. PAGINACIÓN
20. Administración	/admin/users/{id}	DELETE	Suspender o eliminar un usuario
21. Administración	/admin/posts/{id}	DELETE	Eliminar contenido inapropiado (moderación manual)
22. Administración	/admin/reports	GET	Consultar denuncias o contenidos reportados
23. Administración	/admin/stats	GET	Obtener estadísticas de uso del sistema

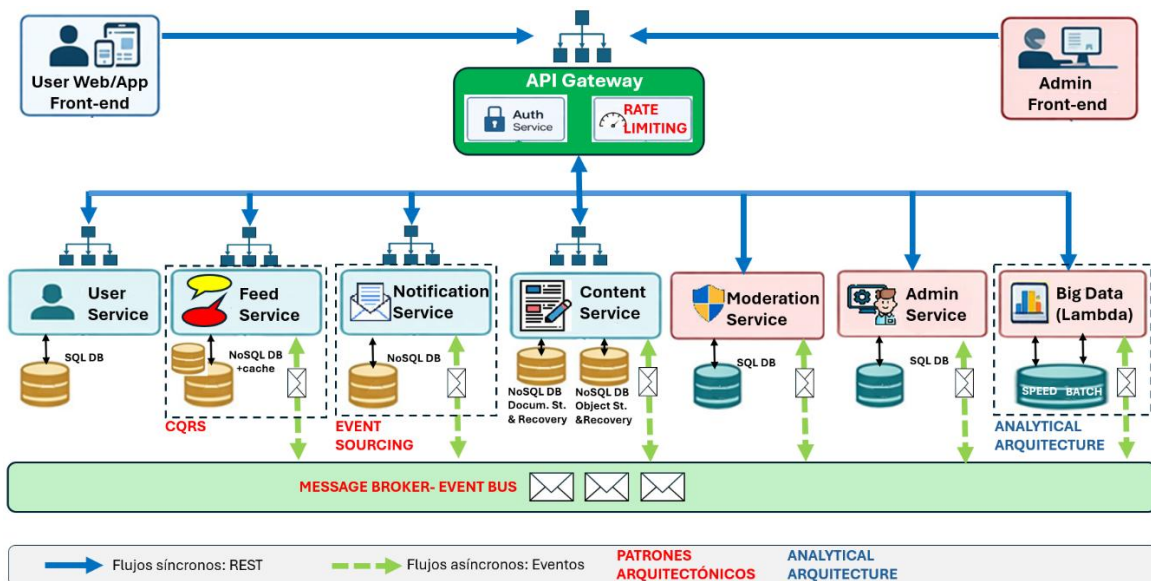
3.3. EJEMPLO DE INFORMACIÓN DE RECURSOS (formato JSON)

La información completa de los recursos en formato JSON se publica en el ANEXO A. Se incluye aquí, a modo ilustrativo, algunos ejemplos de recursos destacados.

8. Publicaciones – POST /posts <pre>{ "id": "post-123", "authorId": "user-123", "content": { "text": "Mi primera publicación", "media": [{ "type": "image", "url": "https://example.com/image.jpg" }] }, "visibility": "public", "createdAt": "2026-04-10T18:25:43Z" }</pre>	6. Relaciones – POST /users/{id}/follow <pre>{ "id": "follow-789", "followerId": "user-456", "followedId": "user-123", "createdAt": "2026-04-10T19:00:00Z" }</pre>	19. Administración – GET /admin/users <pre>{ "users": [{ "id": "user-123", "username": "Juanperez", "email": "juan@example.com", "role": "user", "status": "active", "createdAt": "2025-12-01T10:15:30Z", "lastLogin": "2026-04-10T18:00:00Z", "stats": { "posts": 34, "followers": 120, "following": 75 } }], "pagination": { "page": 1, "pageSize": 20, "total": 150, "hasMore": true } }</pre>
--	--	---

4. VISTA GENERAL DE LA ARQUITECTURA (MODELO ARQUITECTÓNICO)

Se recoge en este gráfico conceptual los principales elementos y conectores del modelo arquitectónico elegido para este proyecto.



5. COMPONENTES DE LA ARQUITECTURA Y DECISIONES DE DISEÑO

5.1. VISIÓN GLOBAL

La solución adopta una arquitectura de microservicios desplegada en la nube, donde el sistema se descompone en componentes independientes alineados con los dominios funcionales principales: autenticación, usuarios, contenido, feed, notificaciones, moderación, administración y analítica. Esta descomposición responde a los drivers arquitectónicos identificados, permitiendo escalar de forma independiente los servicios más críticos (contenido, feed y notificaciones). Se combinan comunicaciones sincrónicas (REST) para operaciones del flujo principal del usuario (autenticación, consulta de perfil, publicación) con comunicaciones asíncronas basadas en eventos mediante un Message Broker, utilizado para notificaciones, generación del feed y analítica. Este enfoque mejora la escalabilidad y el desacoplamiento del sistema.

5.2. API GATEWAY

El sistema dispone de un API Gateway como punto único de entrada, encargado del enrutamiento de peticiones, validación de autenticación (JWT) y aplicación de políticas de control de acceso (rate limiting). Esta decisión simplifica la interacción con clientes y centraliza aspectos transversales como seguridad y control de tráfico.

5.3. USER SERVICE

Gestiona perfiles de usuario y relaciones (seguimiento), dando soporte a RFU.3 y RFU.4. Utiliza una base de datos relacional (SQL) para garantizar consistencia fuerte en los datos de identidad. Se comunica de forma síncrona mediante REST y emite eventos (por ejemplo, “UserFollowed”) para su consumo por otros servicios.

5.4. CONTENT SERVICE

Responsable de la gestión de publicaciones (RFU.5), caracterizadas por un alto volumen de escritura y datos heterogéneos. Se emplea almacenamiento NoSQL tipo Document Store para contenido general con un puntero URL de los contenidos multimedia hacia otra base de datos NoSQL Object Store especializada. Incluye sistema recovery para poder regenerar toda la información en caso de caída de sistema. Tras la creación de contenido, se generan eventos (por ejemplo, “PostCreated”) que permiten actualizar otros servicios sin acoplamiento directo.

5.5. FEED SERVICE

Cuando un usuario crea una publicación, el Content Service emite un evento y el Feed Service consume este evento y actualiza los feeds de los seguidores del autor. Esta actualización puede tener una latencia de segundos (consistencia eventual), aceptable para redes sociales (RFU.6). Es un servicio intensivo en lectura, optimizado mediante caché y separación de responsabilidades respecto al Content Service (enfoque similar a CQRS). El feed se actualiza a partir de eventos, adoptando un modelo de consistencia eventual para mejorar el rendimiento.

5.6. NOTIFICATION SERVICE

Implementa el sistema de notificaciones (RFU.9) mediante una arquitectura basada en eventos. Recibe eventos como “PostCreated”, “PostLiked” o “CommentCreated” y genera notificaciones en tiempo cercano al real. Se emplea almacenamiento de eventos para mantener histórico y mejorar trazabilidad.

5.7. MODERATION SERVICE

Gestiona la revisión de contenidos (RFA.2) y da soporte al cumplimiento normativo (DSA). Su aislamiento permite evolucionar las políticas de moderación e integrar mecanismos automáticos sin afectar al resto del sistema.

5.8. ADMIN SERVICE

Proporciona funcionalidades de administración (RFA.1, RFA.3). Se separa del resto de servicios para reforzar la seguridad y diferenciar claramente los flujos administrativos de los de usuario.

5.9. ANALYTICS ARCHITECTURE

No es un microservicio en sí, sino una arquitectura para Big Data para el procesamiento de métricas y estadísticas (RFA.6). Consume eventos generados por otros microservicios y utiliza mecanismos Speed/Batch orientado a consultas analíticas tanto en tiempo real como en batch, evitando impacto en el rendimiento del sistema principal.

5.10. CONECTORES Y COMUNICACIÓN

Se emplean dos tipos de comunicación:

Síncrona (REST): entre cliente, API Gateway y servicios para operaciones críticas.

Asíncrona (eventos): mediante un Message Broker (pub/sub), utilizado para propagar eventos como “PostCreated” o “UserFollowed”.

Este enfoque permite desacoplar los servicios y mejorar la escalabilidad y tolerancia a fallos.

5.11. PERSISTENCIA Y CONSISTENCIA

Cada microservicio gestiona su propia base de datos, seleccionada según sus necesidades:

SQL → usuarios, administración y moderación (consistencia)

Object Store → contenido multimedia

Cache + NoSQL → feed (rendimiento)

Event Store → notificaciones

NoSQL → analítica

Se adopta un modelo híbrido de consistencia:

Fuerte → autenticación y usuarios

Eventual → feed, notificaciones y analítica

5.12. TRADE-OFFS

La arquitectura permite alta escalabilidad, disponibilidad y desacoplamiento, a costa de una mayor complejidad y la introducción de consistencia eventual en algunos componentes. Se priorizan los atributos de calidad clave (escalabilidad, disponibilidad y rendimiento) frente a la simplicidad del sistema.

6. CONCLUSIÓN

La arquitectura diseñada se basa en un modelo de microservicios que descompone el sistema en componentes independientes alineados con los principales dominios funcionales de la red social. Este enfoque permite abordar de forma efectiva los drivers arquitectónicos identificados, especialmente aquellos relacionados con la publicación de contenido, la interacción entre usuarios y el sistema de notificaciones. El uso combinado de comunicación síncrona (REST) para operaciones críticas y la comunicación asíncrona basada en eventos mediante un Message Broker permite mejorar el desacoplamiento entre servicios y soportar escenarios de alta concurrencia. Asimismo, la separación de responsabilidades (por ejemplo, entre Content Service y Feed Service) y el uso de mecanismos como caché y procesamiento basado en eventos contribuyen a optimizar el rendimiento del sistema.

La arquitectura prioriza la escalabilidad, la disponibilidad y el rendimiento, adoptando en ciertos casos un modelo de consistencia eventual como compromiso necesario en sistemas distribuidos. Esta decisión implica un aumento de la complejidad, pero permite responder de forma más adecuada a los requisitos de una plataforma social a gran escala.

En conjunto, la solución propuesta ofrece una base flexible, escalable y preparada para evolucionar, alineada con los objetivos estratégicos del sistema.

ANEXO A. INFORMACIÓN COMPLETA DE RECURSOS

1. Autenticación – POST /auth/register

```
{
  "id": "user-123",
  "username": "juanperez",
  "email": "juan@example.com",
  "role": "user",
  "status": "active",
  "createdAt": "2026-04-10T10:00:00Z"
}
```

2. Autenticación – POST /auth/login

```
{
  "token": "jwt-token",
  "expiresIn": 3600,
  "user": {
    "id": "user-123",
    "username": "juanperez",
    "role": "user"
  }
}
```

3. Usuario – GET /users/{id}

```
{
  "id": "user-123",
  "username": "juanperez",
  "email": "juan@example.com",
  "name": "Juan Pérez",
  "bio": "Amante de la tecnología",
  "profileImage": "https://example.com/avatar.jpg",
  "role": "user",
  "status": "active",
  "createdAt": "2025-12-01T10:15:30Z",
  "lastLogin": "2026-04-10T18:00:00Z",
  "stats": {
    "posts": 34,
    "followers": 120,
    "following": 75
  }
}
```

4. Usuario – PUT /users/{id}

```
{
  "id": "user-123",
  "bio": "Nuevo bio actualizado",
  "profileImage": "https://example.com/avatar2.jpg",
  "updatedAt": "2026-04-10T18:20:00Z"
}
```

5. Usuario – DELETE /users/{id}

```
{
  "id": "user-123",
  "status": "deleted",
  "deletedAt": "2026-04-10T18:22:00Z"
}
```

6. Relaciones – POST /users/{id}/follow

```
{
  "id": "follow-789",
  "followerId": "user-456",
  "followedId": "user-123",
  "createdAt": "2026-04-10T19:00:00Z"
}
```

7. Relaciones – DELETE /users/{id}/follow

```
{
  "id": "follow-789",
  "status": "removed",
  "updatedAt": "2026-04-10T19:10:00Z"
}
```

8. Publicaciones – POST /posts

```
{
  "id": "post-123",
  "authorId": "user-123",
  "content": {
    "text": "Mi primera publicación",
    "media": [
      {
        "type": "image",
        "url": "https://example.com/image.jpg"
      }
    ]
  },
  "visibility": "public",
  "createdAt": "2026-04-10T18:25:43Z"
}
```

9. Publicaciones – GET /posts/{id}

```
{
  "id": "post-123",
  "author": {
    "id": "user-123",
    "username": "juanperez"
  },
  "content": {
    "text": "Mi primera publicación",
    "media": [
      {
        "type": "image",
        "url": "https://example.com/image.jpg"
      }
    ]
  },
  "stats": {
    "likes": 10,
    "comments": 2,
    "shares": 1
  },
  "createdAt": "2026-04-10T18:25:43Z"
}
```

10. Publicaciones – PUT /posts/{id}

```
{
  "id": "post-123",
  "content": {
    "text": "Publicación editada"
  },
  "updatedAt": "2026-04-10T18:40:00Z"
}
```

11. Publicaciones – DELETE /posts/{id}

```
{
  "id": "post-123",
  "status": "deleted",
  "deletedAt": "2026-04-10T18:45:00Z"
}
```

12. Feed – GET /feed

```
{
  "posts": [
    {
      "id": "post-123",
      "author": {
        "id": "user-123",
        "username": "juanperez"
      },
      "content": {
        "text": "Post en el feed"
      },
      "createdAt": "2026-04-10T18:25:43Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 10,
    "total": 100,
    "hasMore": true
  }
}
```

13. Likes – POST /posts/{id}/like

```
{
  "id": "like-456",
  "postId": "post-123",
  "userId": "user-456",
  "createdAt": "2026-04-10T18:26:00Z"
}
```

14. Likes – DELETE /posts/{id}/like

```
{
  "id": "like-456",
  "status": "removed",
  "updatedAt": "2026-04-10T18:28:00Z"
}
```

15. Comentarios – POST /posts/{id}/comments

```
{
  "id": "comment-1",
  "postId": "post-123",
  "authorId": "user-456",
  "text": "Gran publicación!",
  "createdAt": "2026-04-10T18:30:00Z"
}
```

16. Comentarios – GET /posts/{id}/comments

```
{
  "comments": [
    {
      "id": "comment-1",
      "authorId": "user-456",
      "text": "Gran publicación!",
      "createdAt": "2026-04-10T18:30:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 5,
    "total": 20,
    "hasMore": true
  }
}
```

17. Notificaciones – GET /notifications

```
{
  "notifications": [
    {
      "id": "notif-1",
      "type": "like",
      "message": "A alguien le gustó tu publicación",
      "read": false,
      "createdAt": "2026-04-10T18:35:00Z"
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 10,
    "total": 30,
    "hasMore": true
  }
}
```

18. Notificaciones – PUT /notifications/{id}/read

```
{
  "id": "notif-1",
  "read": true,
  "updatedAt": "2026-04-10T18:40:00Z"
}
```

19. Administración – GET /admin/users

```
{
  "users": [
    {
      "id": "user-123",
      "username": "juanperez",
      "email": "juan@example.com",
      "role": "user",
      "status": "active",
      "createdAt": "2025-12-01T10:15:30Z",
      "lastLogin": "2026-04-10T18:00:00Z",
      "stats": {
        "posts": 34,
        "followers": 120,
        "following": 75
      }
    }
  ],
  "pagination": {
    "page": 1,
    "pageSize": 20,
    "total": 150,
    "hasMore": true
  }
}
```

23. Administración – GET /admin/stats

```
{
  "totalUsers": 10000,
  "totalPosts": 50000,
  "activeUsers": 3000,
  "reportsPending": 25
}
```

20. Administración – DELETE /admin/users/{id}

```
{
  "id": "user-123",
  "status": "removed",
  "updatedAt": "2026-04-10T18:50:00Z"
}
```

21. Administración – DELETE /admin/posts/{id}

```
{
  "id": "post-123",
  "status": "removed",
  "reason": "Inappropriate content",
  "updatedAt": "2026-04-10T18:55:00Z"
}
```

22. Administración – GET /admin/reports

```
{
  "reports": [
    {
      "id": "report-1",
      "postId": "post-123",
      "reportedBy": "user-456",
      "reason": "Spam",
      "status": "pending",
      "createdAt": "2026-04-10T18:45:00Z"
    }
  ]
}
```